LECTURE 8: SCALABLE INDEXING for BIG DATA PROCESSING

DS 8001

Table of Contents

1. Hash functions
2. Scalable Indexing
   - Metric inverted files
   - Prefix permutation index
3. Distributed search
4. Practical perspective

Toronto Metropolitan University

WHY HASH? A STORAGE PROBLEM

- A company needs to store and randomly access 100,000 Visa card numbers (16 digits)
- 10^{16} possible 16-digit numbers
- How much storage is required?



- Option 1: Create an array, put all possible Visa numbers into it.
  (Even) 1 byte per array entry $\Rightarrow 10^{16}$ bytes $= 10^4$ TB!

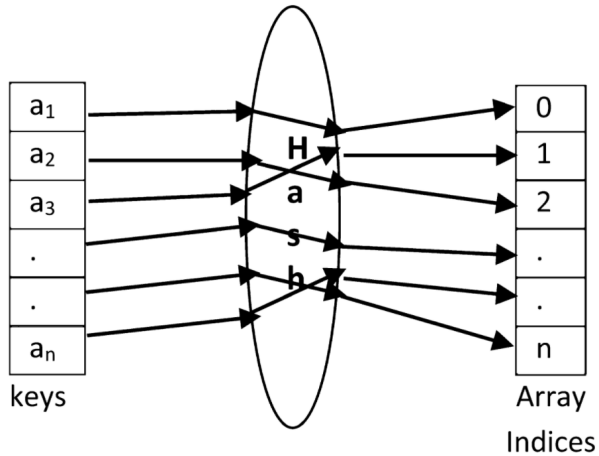
  We need only $10^{-9}\%$ of the entries! $\left(10^5/10^{16}\right)$

- Option 2: Create an array of size 100,000 $\Rightarrow$ 100 KB (1 B/entry)
  - Array indices 0-99999
  - Need a way to map the 16-digit key to an index in $\{0, \ldots, 99999\}$

HASH FUNCTIONS

Hashing

- A **hash function** maps keys from a large (possibly unbounded) set into a small fixed range of integers $\{0, \ldots, m-1\}$.
- The input of a hash function is called a **key**.
- The values returned by a hash function are called **hash values**, **hash codes**, **hash sums**, **checksums**, or simply **hashes**.
- A hash function is *deterministic*, and the array it indexes is the **hash table**.



A FIRST ATTEMPT: THE DIGIT SUM

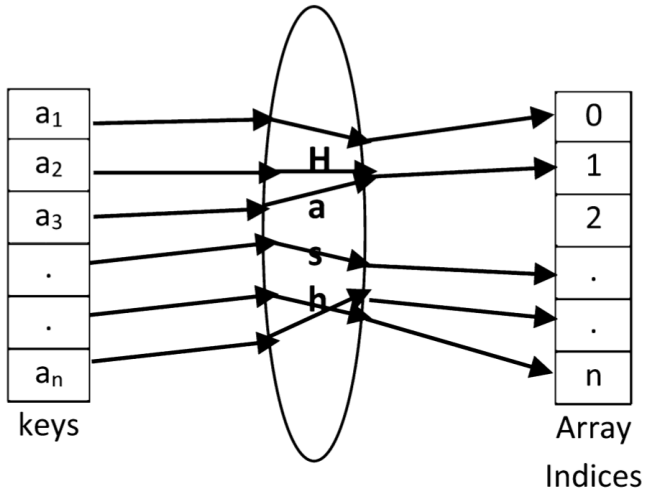
- Let the array index of a card number be the sum of its digits.

$$4000\ 1234\ 5678\ 9010 \longrightarrow 50$$

- What can go wrong?

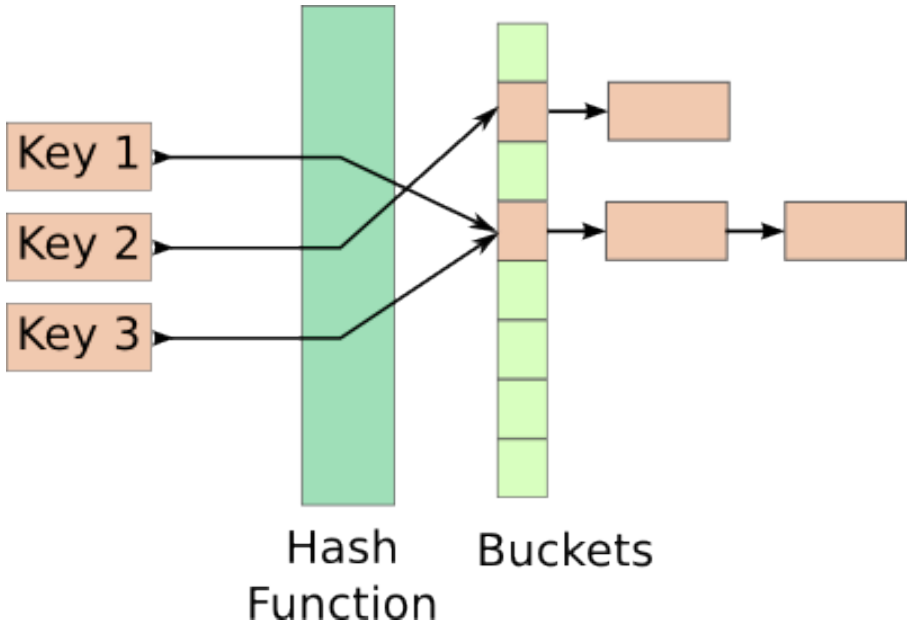
$$3510\ 7822\ 4037\ 2222 \longrightarrow 50$$

- Two keys share a slot: a **collision**. Also, a 16-digit digit sum lies in $\{0, \ldots, 144\}$, so only 145 of the 100000 slots are ever used.



COLLISION

- Two distinct keys are mapped to the same location, i.e., the same hash value
- *Simple solution* (**separate chaining**): Store in that location a pointer to a much smaller array (a **bucket**) that contains the information for those few colliding keys



- Collisions in large sets are generally inevitable (pigeonhole principle)
- Fewer collisions are better: a lookup scans the whole bucket

MINIMIZING COLLISIONS

- Break up the Visa number, x, into a sequence of smaller numbers: Four 4-digit numbers (x_1, x_2, x_3, x_4)
- Suppose $h(x) = h(x_1, x_2, x_3, x_4)$ maps the Visa number into a smaller number, an index $< 100,000$
- If we are given two distinct Visa numbers

$$x' = (x'_1, x'_2, x'_3, x'_4) \text{ and } x'' = (x''_1, x''_2, x''_3, x''_4)$$

  we want $\Pr\{h(x') = h(x'')\}$ to be very small
- Where is the randomness? Not in the data: x', x'' are *fixed*. We draw *h at random* from a family of hash functions, and the probability is over that draw.
- How to choose the family?

Use mod function and prime numbers

Toronto Metropolitan University

REDUCING COLLISION PROBABILITY

Suppose x_i, $i = 1, \ldots, 4$ are four 4-digit numbers within 16-digit x.

Theorem

Let n be a prime number, and let $0 \leq x_i < n$. Let a_i, $i = 1, \ldots, 4$ be integers chosen independently and uniformly at random from $\{0, 1, \ldots, n-1\}$. Define

$$h(x) = \left(\sum_{i=1}^{4} a_i x_i \right) \bmod n$$

Then, given any two possible distinct values of x, say, x' and x'',

$$\Pr_{a_1, \ldots, a_4} \{h(x') = h(x'')\} = \frac{1}{n}$$

$\therefore$ Using a prime number $n > 100,000$, we can reduce the probability of a collision to less than $1/100,000$. Quite impressive! Four-digit blocks satisfy $x_i \leq 9999 < n$, as required. A family of hash functions with this property is called a *universal hash family*.

Toronto Metropolitan University

EXAMPLE: EVALUATING h_a

n	a_1	a_2	a_3	a_4
100,103	32,217	99,266	36,092	85,585

$$h_a(x) = (32217x_1 + 99266x_2 + 36092x_3 + 85585x_4) \bmod 100103$$

- $x' = 4000\ 1234\ 5678\ 9010$: $h_a(x') = 1227413470 \bmod 100103 = 50587$
- $x'' = 3510\ 7822\ 4037\ 2222$: $h_a(x'') = 1225413596 \bmod 100103 = 52773$

Does the method of calculating the hash function $h_a(x)$ matter? No, since $(u + v) \bmod n = \big((u \bmod n) + (v \bmod n)\big) \bmod n$:

$$\begin{aligned} h_a(x'') = \Big(&113081670 \bmod 100103 + 776458652 \bmod 100103 + \\ &145703404 \bmod 100103 + 190169870 \bmod 100103\Big) \bmod 100103 \\ = &(65383 + 59784 + 53539 + 74273) \bmod 100103 = 52773 \end{aligned}$$

Reducing term by term keeps all intermediate values below n^2, however many blocks are summed: no overflow in 64-bit arithmetic.

PROOF OF THE THEOREM

$$h(x) = \left(\sum_{i=1}^{4} a_i x_i \right) \bmod n$$

Given $x' \neq x''$, we want to show $\Pr\left\{h(x') = h(x'')\right\} = 1/n$.

Since $x' \neq x''$, assume, WLOG, $x'_4 \neq x''_4$. Say we already picked a_1, a_2, and a_3, and we are about to draw a_4.

$$h(x') \equiv h(x'') \pmod{n}$$

$$\sum_{i=1}^{3} a_i x'_i + a_4 x'_4 \equiv \sum_{i=1}^{3} a_i x''_i + a_4 x''_4 \pmod{n}$$

$$\underbrace{\sum_{i=1}^{3} a_i (x'_i - x''_i)}_{c} \equiv a_4 \underbrace{(x''_4 - x'_4)}_{b} \pmod{n}$$

As $0 < |x'_4 - x''_4| < n$ and n is prime, $b \not\equiv 0$ and b^{-1} exists mod n. Hence a_4 must be precisely $cb^{-1} \pmod{n}$, exactly one out of its n equally likely values, and a_4 is drawn independently of $a_1, a_2, a_3 \Rightarrow$ probability $1/n$. $\square$

CONCEPT CHECK: WHY A PRIME MODULUS?

- **Q.** The proof needs b^{-1} to exist modulo n. What breaks if n is composite?
- **A.** b^{-1} exists only if $\gcd(b, n) = 1$. Take $n = 10$ and two keys differing only in the last block, with $b = x_4'' - x_4' = 5$ (so $c = 0$). Then $h(x') = h(x'')$ whenever $5a_4 \equiv 0 \pmod{10}$, i.e. for $a_4 \in \{0, 2, 4, 6, 8\}$:

$$\Pr\{h(x') = h(x'')\} = \tfrac{5}{10} \gg \tfrac{1}{n}$$

- **Q.** With $n = 100103$, can h_a be used directly as an index into the 100000-slot table?
- **A.** No: h_a returns values in $\{0, \ldots, 100102\}$. Either size the table n, or reduce mod 100000 afterwards — the theorem bounds collisions of h_a, not of the final index.

Scalable Indexing

K-NN SEARCH PROBLEM

- **Reference**: H Mohamed and S Marchand-Maillet, "*Scalable indexing for big data processing*", in *Big Data: Algorithms, Analytics, and Applications*, Chapman & Hall/CRC, 2015.
- Part 1 indexed keys for *exact-match* lookup; the rest of the lecture indexes objects for *similarity* search.
- K-NN search: given a query q, a database D and a distance d, return the K objects of D closest to q.
  - *Applications*: Information retrieval and visualization, machine learning, and data mining.
- At big-data scale exact K-NN is too costly; we accept *approximate* answers, which contain most, but not necessarily all, of the true K neighbours.
- Permutation-based indexing (PBI) is a method for approximate similarity search in large-scale domains: an object is encoded by the *order* in which fixed reference objects (**pivots**) surround it.
- Part 3 distributes the index and the search over P machines.

Toronto Metropolitan University

SIMILARITY SEARCH

- **Goal**: extract the most similar objects to a given query.
- Used in:
  - Genome comparison to find similarities between genes
  - Multimedia retrieval to find a similar picture or video
- Three families of implementations:
  - **Exhaustive search**: For each query, distance between the query object and *every* database object is calculated while keeping the similar/close objects
    $\Rightarrow$ Problem: it does not scale with large collections ($O(N)$ distances per query)
  - **Exact search**: By using Space Decomposition techniques (metric trees prune subtrees via the triangle inequality), the number of objects compared to the query object is reduced.
    $\Rightarrow$ Problem: not efficient for high dimensional data sets — the *curse of dimensionality*: distances concentrate, little can be pruned
  - **Approximate search**: This method reduces the amount of data that needs to be accessed by using some hashing, Space Partitioning (using pivots) or Space Transformation techniques. It provides fast and scalable response time.
    $\Rightarrow$ Limitation: some imprecisions in the result — true neighbours may be missed; quality is measured by *recall*

APPROXIMATE SEARCH TECHNIQUES

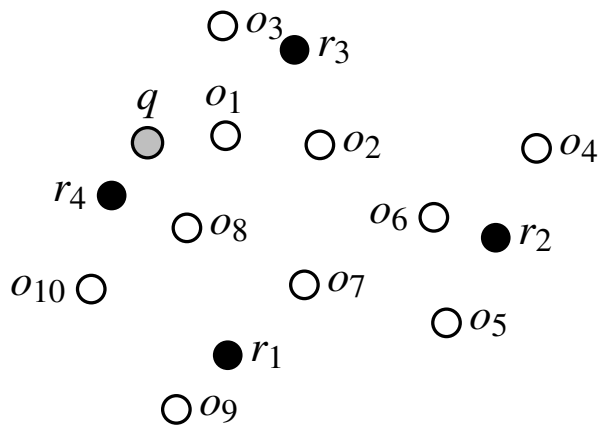
Each technique trades some accuracy for speed:

- Locality-sensitive hashing (LSH) — hashing; nearby objects collide with high probability (Part 1's tool, but here collisions are wanted)
- FastMap — space transformation; embeds objects in a low-dimensional Euclidean space
- M-tree — space decomposition; an *exact* metric tree, made approximate by stopping early
- Permutation-based indexing (PBI) — space partitioning by pivots; ranks by the *order* of pivots, never by distance values

Only PBI is developed below.

PERMUTATION BASED INDEXING (PBI)

- For every database object, PBI stores the reference objects (pivots) sorted by their distance to that object
- That order permutation is the object's encoding: two objects are similar when the pivots are ordered similarly around them; no distance *value* is compared
- R is typically a small random sample of D ($n = |R| \ll N = |D|$); indexing costs n distances per object, once, offline
- The technique is efficient. However, with the exponential increase of the database size, parallel and distributed computing is necessary



○ data objects o_i (white)

● reference objects, pivots r_j (black)

◉ query q (grey)

Running example, reused throughout:
$N = 10$ objects, $n = 4$ pivots (drawn from D but listed apart from the o_i).

Ordered List

Definition

Given a set of N objects o_i, $D = \{o_1, \ldots, o_N\}$ in m-dimensional space, a set of reference objects $R = \{r_1, \ldots, r_n\} \subset D$, and a distance function that follows the metric space postulates (positivity, symmetry, triangle inequality), we define the ordered list of R relative to $o \in D$, $L(o, R)$, as the ordering of elements in R with respect to their increasing distance from o:

$$L(o, R) = (r_{i_1}, \ldots, r_{i_n}) \text{ such that } d(o, r_{i_j}) \leq d(o, r_{i_{j+1}}), \forall j = 1, \ldots, n-1$$

(L is a *tuple*, not a set; ties broken by pivot index.)
Then for any $r \in R$, $L(o, R)_{|r}$ indicates the position of r in $L(o, R)$, i.e., $L(o, R)_{|r} = j$ such that $r_{i_j} = r$. Further, given $n \geq \tilde{n} > 0$, $\tilde{L}(o, R)$ is the pruned ordered list of the $\tilde{n}$ first elements of $L(o, R)$; $\tilde{L}(o, R)_{|r}$ is defined only for the $\tilde{n}$ surviving pivots.

$L(o_1, R) = (r_3, r_4, r_1, r_2)$ $L(o_2, R) = (r_3, r_2, r_4, r_1)$ $L(o_3, R) = (r_3, r_4, r_1, r_2)$
$L(o_4, R) = (r_2, r_3, r_1, r_4)$ $L(o_5, R) = (r_2, r_1, r_3, r_4)$ $L(o_6, R) = (r_2, r_3, r_1, r_4)$
$L(o_7, R) = (r_1, r_2, r_4, r_3)$ $L(o_8, R) = (r_4, r_1, r_3, r_2)$ $L(o_9, R) = (r_1, r_4, r_2, r_3)$
$L(o_{10}, R) = (r_4, r_1, r_3, r_2)$ $L(q, R) = (r_4, r_3, r_1, r_2)$

Full lists of the running example ($n = 4$). The first $\tilde{n} = 2$ entries (blue) are the pruned lists $\tilde{L}(\cdot, R)$: all that the index stores.

SIMILARITY

- In a K-NN query we are interested only in *which* objects come first, not in the distance values
- PBI therefore compares the ordered lists of q and o_i instead of computing $d(q, o_i)$
- **Spearman footrule distance** (d_{SFD}) is used to measure the dissimilarity between ordered lists

$$d(q,o_i) \simeq^{\text{rank}} d_{SFD}(q,o_i) = \sum_{r_j \in \tilde{L}(q,R) \cap \tilde{L}(o_i,R)} |\tilde{L}(q,R)_{|r_j} - \tilde{L}(o_i,R)_{|r_j}|$$

- $\simeq^{\text{rank}}$ means "induces approximately the same *ranking*", not "is numerically close to".
- The sum runs over the *common* pivots, since $\tilde{L}(\cdot)_{|r_j}$ is undefined for a pruned-away pivot. With $\tilde{n} = n$ nothing is pruned and this is the classical Spearman footrule. Ignoring missing pivots has consequences — we return to this when ranking with an index.

SPEARMAN FOOTRULE DISTANCE

- L_1 distance between two rank vectors
- E.g., we ask two people about their food preferences
  - Person A says [vegetables, meat, rice]
  - Person B says [meat, vegetables, rice]
- Each list becomes a rank vector whose *coordinates are indexed by item* (vegetables, meat, rice) and whose entries are that item's *rank*
  - Person A gets the rank vector [1, 2, 3]
  - Person B gets the rank vector [2, 1, 3]

  (B ranks vegetables 2nd, meat 1st, rice 3rd. Do *not* read these vectors as "the list in order".)
- Footrule distance between them is calculated as

$$d_{SFD} = |1-2| + |2-1| + |3-3| = 2$$

- $d_{SFD} = 0$ iff the rankings agree, and its maximum over rankings of n items is $\lfloor n^2/2 \rfloor$ (here 4, for [1,2,3] vs. [3,2,1]) — the natural normaliser.

PBI INDEXING ALGORITHM

Input: D of size N, R of size n and $\tilde{n} \leq n$

Output: $\tilde{L}(o_i, R), \quad \forall\, i = 1, \ldots, N$

for $o_i \in D$ **do**

  for $r_j \in R$ **do**

    $b[j].dis = d(o_i, r_j)$

    $b[j].indx = j$

  end for

  $L(o_i, R) = quicksort(b, n)$

  $\tilde{L}(o_i, R) = partiallist(L(o_i, R), \tilde{n})$ # keep the first $\tilde{n}$ entries

  Store the ID i of o_i and its $\tilde{L}(o_i, R)$ for other processing

end for

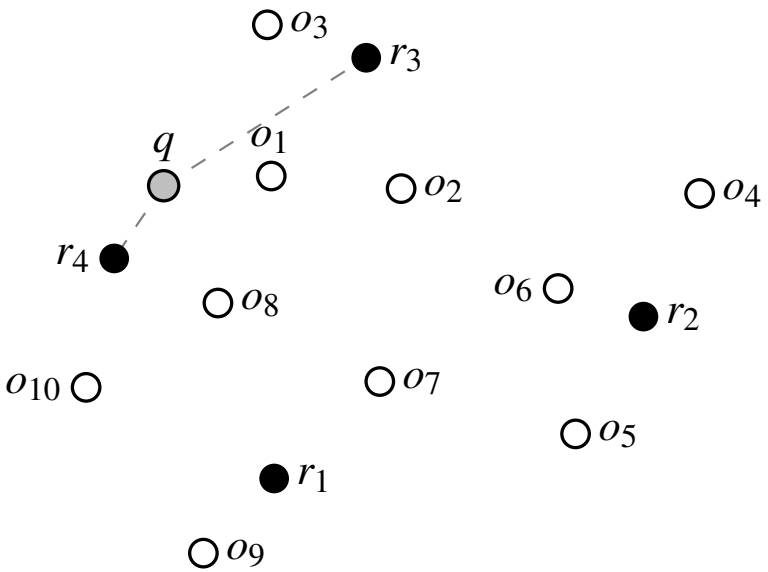
Complexity: $O(N(n + n\log n)) = O(Nn\log n)$: n distances plus one sort of n items per object. Charging $O(m)$ per distance in m dimensions gives $O\big(Nn(m + \log n)\big)$.

THE RETRIEVAL PROBLEM

- After indexing, each object o_i is stored as its pruned list $\tilde{L}(o_i, R)$: its $\tilde{n}$ nearest pivots, in order.
- A query needs only n distance computations to obtain its own list $\tilde{L}(q, R)$.
- **Naive search**: compare $\tilde{L}(q, R)$ with all N stored lists. Each comparison is cheap, but every query still touches the *whole* database.
- Goal: retrieve the objects whose lists resemble $\tilde{L}(q, R)$ *without* looking at all the other lists.
- **Why shared pivots are a good sign**: by the triangle inequality, $|d(q, r_j) - d(o, r_j)| \leq d(q, o)$ for every pivot r_j. An object close to q sees every pivot at almost the same distance as q does, so it tends to rank the pivots the same way.

We need a structure that finds the objects sharing pivots with q *directly*.

RUNNING EXAMPLE: FINDING SIMILAR IMAGES



- A photo collection: each image is a **feature vector**, and d is the Euclidean distance between vectors. (Real image features often have hundreds of dimensions; we draw two.)
- $o_1, \ldots, o_{10}$: images; $r_1, \ldots, r_4$: reference images (pivots); q: the user's photo.
- **Task**: return the $K = 2$ images most similar to q.
- Pruned lists, $\tilde{n} = 2$ (nearest pivot first):

o_1: (r_3, r_4)	o_6: (r_2, r_3)
o_2: (r_3, r_2)	o_7: (r_1, r_2)
o_3: (r_3, r_4)	o_8: (r_4, r_1)
o_4: (r_2, r_3)	o_9: (r_1, r_4)
o_5: (r_2, r_1)	o_{10}: (r_4, r_1)

q: (r_4, r_3) (dashed lines in the figure)

Analogy: a search engine's index

Three tiny documents: **D1** "cats chase mice", **D2** "dogs chase cats", **D3** "mice eat cheese".

Inverted index

cats $\rightarrow$ D1, D2
chase $\rightarrow$ D1, D2
cheese $\rightarrow$ D3
dogs $\rightarrow$ D2
eat $\rightarrow$ D3
mice $\rightarrow$ D1, D3

Query "dogs cats"

1. Read only two lists:
   dogs $\rightarrow$ D2; cats $\rightarrow$ D1, D2
2. Candidates: D1, D2. D3 is never touched.
3. Rank: D2 contains both words, D1 only one
   $\Rightarrow$ D2 first.

The idea to borrow

For every "word", store the list of items that contain it. A query reads only the lists of its own words, so items sharing no word with the query are never touched.

MIF: PIVOTS PLAY THE ROLE OF WORDS

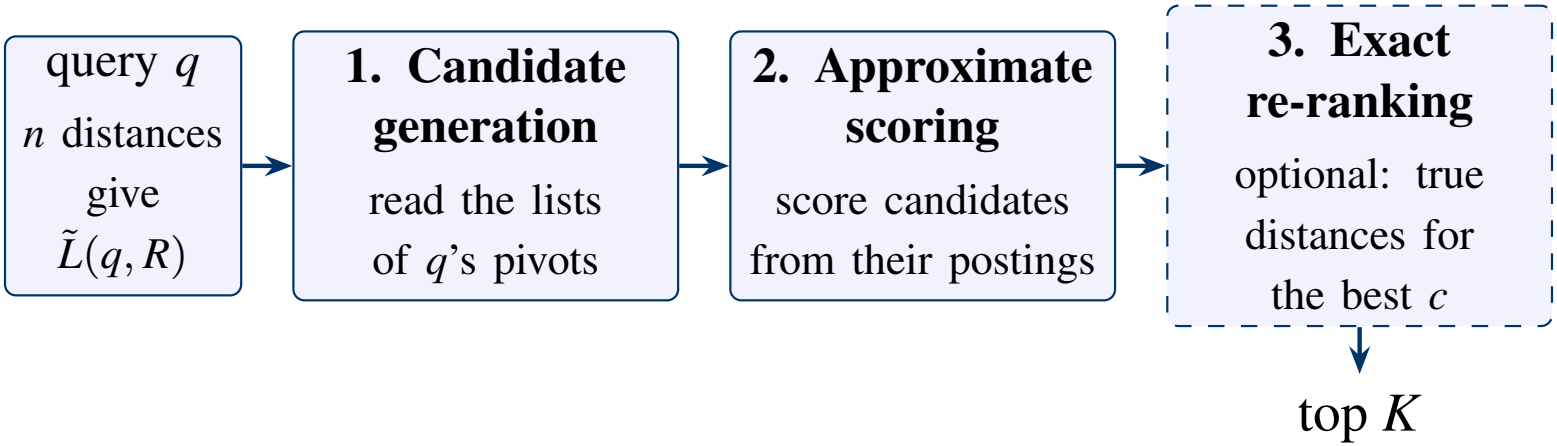
Text search		**Metric inverted file**
word	$\leftrightarrow$	pivot r_j
document	$\leftrightarrow$	object o_i
word occurs in document	$\leftrightarrow$	r_j appears in $\tilde{L}(o_i, R)$
posting: document ID	$\leftrightarrow$	posting: $(o_i,$ rank of r_j in $\tilde{L}(o_i, R))$

Building it: each pruned list adds one posting to the list of every pivot it names. E.g. $\tilde{L}(o_1, R) = (r_3, r_4)$ adds $(o_1, 1)$ to the list of r_3 and $(o_1, 2)$ to the list of r_4.

$$
\begin{aligned}
r_1 &\rightarrow (o_5,2)\ (o_7,1)\ (o_8,2)\ (o_9,1)\ (o_{10},2)\\
r_2 &\rightarrow (o_2,2)\ (o_4,1)\ (o_5,1)\ (o_6,1)\ (o_7,2)\\
r_3 &\rightarrow (o_1,1)\ (o_2,1)\ (o_3,1)\ (o_4,2)\ (o_6,2)\\
r_4 &\rightarrow (o_1,2)\ (o_3,2)\ (o_8,1)\ (o_9,2)\ (o_{10},1)
\end{aligned}
$$

The index holds $N\tilde{n}$ postings (here 20). It is built once, offline; the lists can stay on disk, because a query reads each list from start to end.

ANSWERING A QUERY IN THREE STAGES



- Stages 1 and 2 compute *no* distances: they only read and add up postings.
- Stage 3 computes c true distances ($c \ll N$), after fetching those c feature vectors.
- Each stage works only with what the previous one passed on: re-ranking can reorder the candidates, but an object missed in stage 1 never comes back.
- When the index is spread over several machines, a fourth step, **result aggregation**, combines their answers (Part 3).

Stage 1: candidate generation

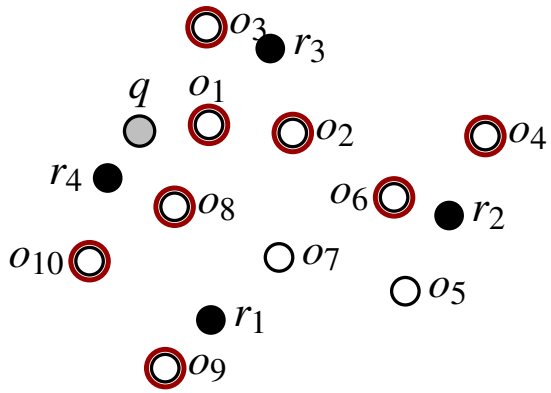
$\tilde{L}(q, R) = (r_4, r_3)$, so we read **only** the lists of r_4 and r_3:

$r_1 \rightarrow$ $(o_5, 2)$ $(o_7, 1)$ $(o_8, 2)$ $(o_9, 1)$ $(o_{10}, 2)$

$r_2 \rightarrow$ $(o_2, 2)$ $(o_4, 1)$ $(o_5, 1)$ $(o_6, 1)$ $(o_7, 2)$

$r_3 \rightarrow$ $(o_1, 1)$ $(o_2, 1)$ $(o_3, 1)$ $(o_4, 2)$ $(o_6, 2)$

$r_4 \rightarrow$ $(o_1, 2)$ $(o_3, 2)$ $(o_8, 1)$ $(o_9, 2)$ $(o_{10}, 1)$



- Candidates (circled) = union of the two lists: $o_1, o_2, o_3, o_4, o_6, o_8, o_9, o_{10}$.
- o_5 and o_7 share no pivot with q, so they are never touched. We read 10 of the 20 postings.
- **At scale**: a list holds $N\tilde{n}/n$ postings on average, so a query reads about $\tilde{n} \cdot N\tilde{n}/n$ of them — a fraction $\tilde{n}/n$ of the index if lists have similar lengths. E.g. $n = 10\,000$, $\tilde{n} = 10$: 0.1%.

Toronto Metropolitan University

STAGE 2: A TEMPTING SCORE — AND WHY IT FAILS

Tempting: add $|\text{rank in } q - \text{rank in } o|$ over the pivots the two lists *share* (the d_{SFD} of the SIMILARITY slide); smaller is better.

	o_1	o_2	o_3	o_4	o_6	o_8	o_9	o_{10}	o_5	o_7
shared pivots	2	1	2	1	1	1	1	1	0	0
score	2	1	2	0	0	0	1	0	0	0

- o_4 and o_6 share only r_3 and score a perfect 0, ahead of o_1, which shares *both* pivots and is in fact the image nearest to q.
- A pivot missing from one list costs nothing, so *sharing less can look better*. In the extreme, o_5 and o_7, which share nothing, would also score 0 if they were scored.
- The top 4 by this score are o_4, o_6, o_8, o_{10}; o_4 and o_6 are among the farthest images.

A good score must *penalise* pivots that appear in only one of the two lists.

STAGE 2: PENALISE MISSING PIVOTS

Fix: treat a pivot that is absent from a pruned list as if it sat just after the list, at position $\tilde{n}+1$ (the footrule with a *location parameter* of Fagin et al.):

$$\text{score}(q,o) = \sum_{r \in \tilde{L}(q,R) \cup \tilde{L}(o,R)} \left| \text{pos}_q(r) - \text{pos}_o(r) \right|$$

where $\text{pos}_o(r) = \tilde{L}(o,R)_{|r}$ if r is in the list of o, and $\tilde{n}+1$ otherwise (same for q).

Running example: $\tilde{n}=2$, so a missing pivot sits at position 3; $q=(r_4,r_3)$.

$o_1=(r_3,r_4)$: $\underbrace{|1-2|}_{r_4} + \underbrace{|2-1|}_{r_3} = 2$ shares both pivots

$o_4=(r_2,r_3)$: $\underbrace{|1-3|}_{r_4} + \underbrace{|2-2|}_{r_3} + \underbrace{|3-1|}_{r_2} = 4$ shares one

$o_5=(r_2,r_1)$: $\underbrace{|1-3|}_{r_4} + \underbrace{|2-3|}_{r_3} + \underbrace{|3-1|}_{r_2} + \underbrace{|3-2|}_{r_1} = 6$ shares none

- The score is 0 only for identical lists; its maximum, $\tilde{n}(\tilde{n}+1)=6$, is reached exactly when no pivot is shared.
- All objects: $o_1, o_3, o_8, o_{10} \to 2$; $o_2, o_4, o_6, o_9 \to 4$; $o_5, o_7 \to 6$. The nearest image o_1 is back at the top, and objects sharing nothing come last.

STAGE 2: POINTS FROM THE INVERTED LISTS

This score can be computed while reading *only q's lists*:

- Each posting (o, rank) in the list of one of q's pivots earns o $\tilde{n} + 1 - \max\{\text{rank in } q,\ \text{rank in } o\}$ points.
- Add up each candidate's points: **more points = more similar**.
- This gives exactly the ranking of the previous slide, since score $= \tilde{n}(\tilde{n}+1) - 2\cdot$points: a shared pivot replaces two "missing" penalties by one smaller term.

With $\tilde{n} = 2$: **2 points** if the pivot is first in both lists, otherwise **1 point**.

r_4 (1st for q) $\rightarrow$ $(o_1, 2)$ **+1** $(o_3, 2)$ **+1** $(o_8, 1)$ **+2** $(o_9, 2)$ **+1** $(o_{10}, 1)$ **+2**

r_3 (2nd for q) $\rightarrow$ $(o_1, 1)$ **+1** $(o_2, 1)$ **+1** $(o_3, 1)$ **+1** $(o_4, 2)$ **+1** $(o_6, 2)$ **+1**

Totals: o_1, o_3, o_8, o_{10}: **2** points (score 2); o_2, o_4, o_6, o_9: **1** point (score 4); o_5, o_7: never touched (score 6).

Toronto Metropolitan University

STAGE 3: EXACT RE-RANKING

Take the c best candidates ($c = 4$: the whole 2-point group) and compute their *true* Euclidean distances to $q = (1.20,\ 3.95)$:

candidate	o_1	o_8	o_3	o_{10}
feature vector	$(2.31,\ 4.05)$	$(1.76,\ 2.74)$	$(2.27,\ 5.61)$	$(0.40,\ 1.87)$
$d(q, \cdot)$	**1.11**	**1.33**	1.97	2.23

e.g. $d(q, o_1) = \sqrt{(2.31 - 1.20)^2 + (4.05 - 3.95)^2} = \sqrt{1.2321 + 0.0100} \approx 1.11$

- Return the $K = 2$ closest: o_1, o_8. Checking all ten distances confirms that these are the true 2 nearest neighbours.
- **Recall**@K = fraction of the true K nearest neighbours that are returned. Here $2/2$; with the tempting score (top 4: o_4, o_6, o_8, o_{10}) it would be $1/2$.
- Cost: c distance computations instead of N. For a fixed index and score, a larger c can only keep or raise recall — but never beyond what stage 1 found.

THE COMPLETE MIF ALGORITHM AND ITS COST

Build (offline, once)

1. Choose n pivots R, e.g. a random sample of D.
2. For each object o: compute $d(o, r)$ for all $r \in R$; keep the $\tilde{n}$ nearest, in order.
3. For each rank $p \leq \tilde{n}$: append (o, p) to the list of the pivot at rank p.

Query q, return K results

1. Compute $\tilde{L}(q, R)$: n distances.
2. For each pivot r of q, each posting (o, p): points$[o]$ += $\tilde{n} + 1 - \max(\text{pos}_q(r), p)$.
3. Keep the c candidates with most points.
4. Compute $d(q, o)$ for these c; return the K closest.

	build (once)	**per query**
distances	Nn	$n + c$ (exhaustive scan: N)
other work	sorting: $O(Nn \log n)$	read $\approx N\tilde{n}^2/n$ postings*; top c of C candidates: $O(C \log c)$
storage	$N\tilde{n}$ postings + the N vectors	points for the C candidates

* $\tilde{n}$ lists of about $N\tilde{n}/n$ postings each, if lists have similar lengths; C is at most the number of postings read.

At scale ($N = 10^9$, $n = 10^4$, $\tilde{n} = 10$, $c = 10^3$): about 11 000 distances and 10^7 postings per query instead of 10^9 distances.

How long should the pruned lists be?

Same query, $K = 2$, re-ranking the $c = 4$ best candidates:

	$\tilde{n} = 1$	$\tilde{n} = 2$	$\tilde{n} = 3$
postings in the index	10	20	30
postings read by the query	2	10	24
candidates	o_8, o_{10}	8 objects	all 10
returned	o_8, o_{10}	o_1, o_8	o_1, o_8
recall	$1/2$	$2/2$	$2/2$

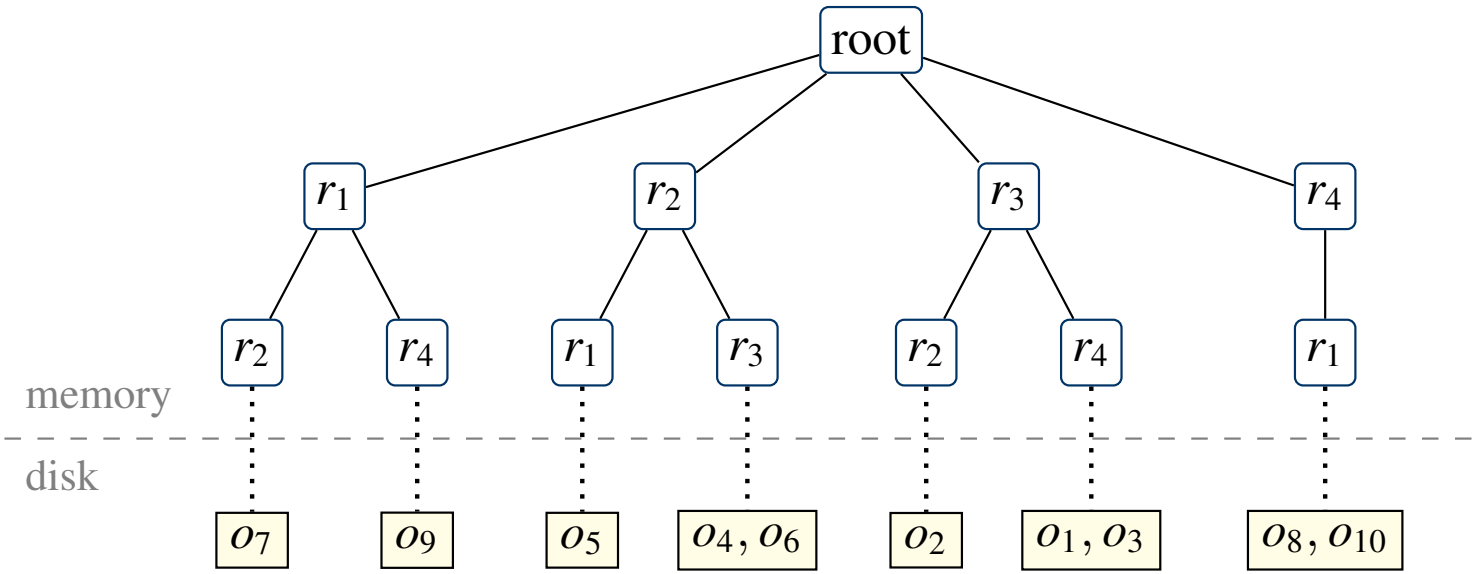
- $\tilde{n} = 1$ misses o_1: its nearest pivot is r_3, while q's is r_4. No re-ranking can repair that.
- Longer lists only *add* candidates, so more true neighbours become reachable. Recall after the top-c cut usually rises too, but not always: the points change, and a true neighbour can drop out of the top c.
- The price: doubling $\tilde{n}$ doubles the index and roughly *quadruples* the postings read. (The toy reads most of its index only because $n = 4$ is tiny.)

QUICK CHECK

- **Q1.** A new image o_{11} has $\tilde{L}(o_{11}, R) = (r_3, r_1)$. Is it a candidate for $q = (r_4, r_3)$? What are its points and its score?
- **A1.** Yes: it shares r_3 (2nd for q, 1st for o_{11}), worth $3 - \max(2, 1) = 1$ point, so its score is $6 - 2 \cdot 1 = 4$, the same as o_2, o_4, o_6, o_9. Check: $|1 - 3|_{r_4} + |2 - 1|_{r_3} + |3 - 2|_{r_1} = 4$.
- **Q2.** The true nearest neighbour of some query shares none of the query's $\tilde{n}$ pivots. Can stage 2 or stage 3 recover it?
- **A2.** No. It is never a candidate, so it is never scored or re-ranked. Only longer lists (a larger $\tilde{n}$) or different pivots can bring it in.
- **Q3.** Why place missing pivots at position $\tilde{n} + 1$ instead of simply ignoring them?
- **A3.** Ignoring them rewards sharing *less* (o_4 beat o_1). At $\tilde{n} + 1$, every missing pivot costs something, and an object scores best only if it shares q's pivots in the same order.

A CONTRASTING DESIGN: THE PREFIX INDEX

- Group the objects by the first ℓ pivots of their permutation, the **prefix** (here $\ell = 2$, so the prefix is exactly the pruned list).
- A small **prefix tree** (trie) in memory leads to the **group** of objects with that prefix, stored side by side on disk.



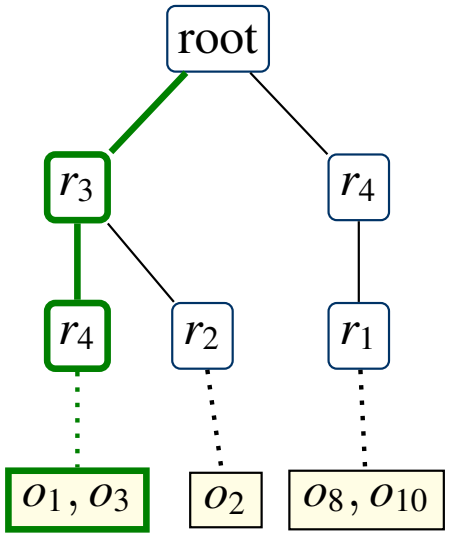
Only the trie is held in memory. Because a group is contiguous, a query typically needs *one* disk read (Esuli, 2009) — as long as the group fits in a few disk blocks; a large group, a back-off to a shorter prefix, or several prefixes mean more reads.

PREFIX INDEX QUERY: A SUCCESSFUL SEARCH

Another user's photo $q' = (2.60,\ 4.90)$ has $L(q', R) = (r_3, r_4, r_1, r_2)$, so its prefix is (r_3, r_4).

1. **Walk the trie** in memory: root $\to r_3 \to r_4$.
2. **Read the group** $\{o_1, o_3\}$: one contiguous disk read.
3. **Re-rank** by true distance: $d(q', o_3) = 0.78$, $d(q', o_1) = 0.90 \Rightarrow$ return o_3, o_1.

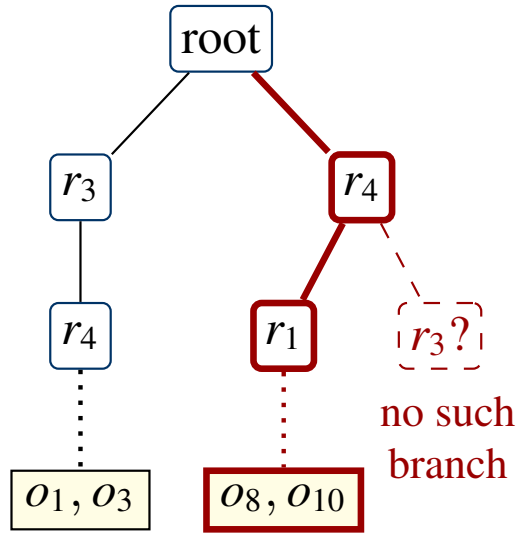
Check: the next closest image, o_2, is at 1.44, so these are the true 2 nearest neighbours (recall $2/2$), found with $n + 2 = 6$ distance computations and one group read.



Part of the trie: the path taken by q' (green).

PREFIX INDEX QUERY: WHEN THE PREFIX IS MISSING

- Now the first query q: follow its prefix (r_4, r_3).
- There is *no* branch $r_4 \to r_3$. Backing off to the shorter prefix (r_4) gives $\{o_8, o_{10}\}$: we return o_8, o_{10} and miss o_1 (recall $1/2$).
- Why: $o_1 = (r_3, r_4)$ has the same two pivots as q in the opposite order. A prefix must match *exactly*, so one swap sends a near neighbour down another branch. The MIF score gives partial credit instead.



Part of the trie: the path taken by q (red), and the branch that holds o_1.

Remedies cost extra reads: shorter prefixes (bigger groups), or searching several prefixes or several indexes per query (Esuli's "boosting"). Here, also reading the group of (r_3, r_4) adds o_1, o_3; re-ranking the four objects returns o_1, o_8.

TWO DESIGNS SIDE BY SIDE

	Metric inverted file	**Prefix index**
copies of an object	$\tilde{n}$ (one posting per list)	1 (in one group)
candidates	objects sharing ≥ 1 pivot with q	objects with exactly q's prefix
ranking	points, then exact re-ranking	exact distances in the group
query cost	read $\tilde{n}$ lists, add up points	trie walk + typically one group read
weak spot	long lists: many postings to read	a swapped prefix goes elsewhere

- **Prefer the MIF** when recall matters and reading several lists is affordable: partial agreement still earns points, so neighbours with a different pivot order are found.
- **Prefer the prefix index** when memory or disk accesses are the bottleneck and lower recall is acceptable, or can be bought back with a few extra prefix searches.
- Storing each object $\tilde{n}$ times is what makes the MIF robust to order changes — and what makes it larger.

Not covered here: the metric suffix array (all pruned lists packed into one in-memory array), the metric permutation table (coarser, bucketed ranks), the brief permutation index (permutations as bit strings).

Distributed search

WHY DISTRIBUTE? A ROUGH ESTIMATE

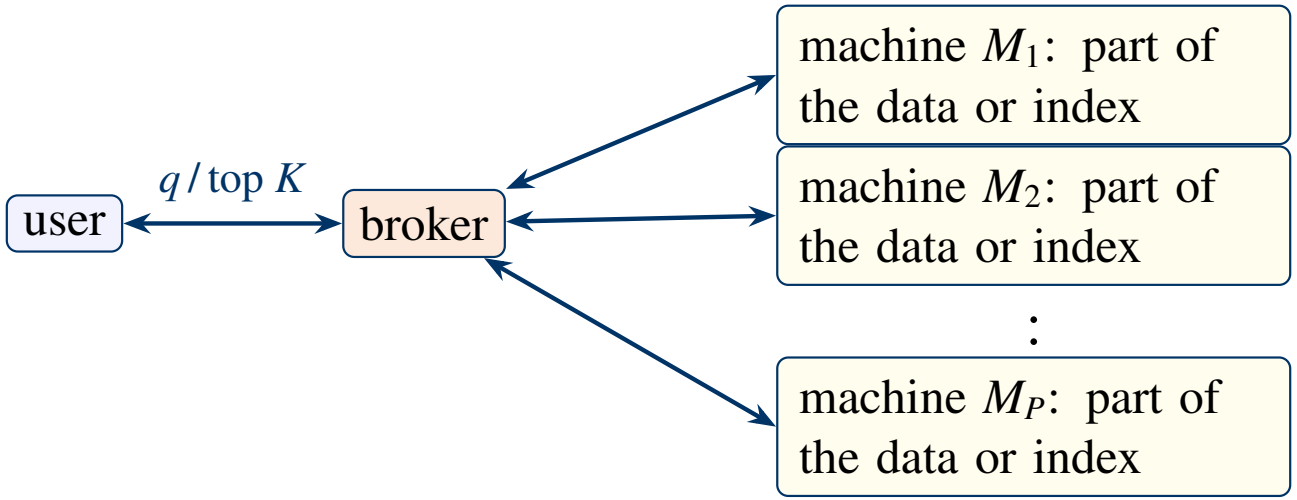
Assume $N = 10^9$ images, 512-dimensional feature vectors (4-byte floats), $n = 10\,000$ pivots and $\tilde{n} = 10$:

feature vectors	$10^9 \times 2\,\text{KB}$	$\approx 2\,\text{TB}$
MIF postings	$N\tilde{n} = 10^{10}$, about 5 bytes each	$\approx 50\,\text{GB}$
indexing work	$N \cdot n = 10^{13}$ distance computations	≈ 28 hours*
work per query	$N\tilde{n}^2/n \approx 10^7$ postings to read	for *every* query

* on one machine, assuming 10^8 distance computations per second. With an ideal speedup, 100 machines would need about 17 minutes; real systems lose part of this to communication, coordination and uneven load.

- On a single machine: terabytes of data, more than a day to build the index, and every query competing for the same disk and processors.
- So we split the collection and/or the index over P machines that work in parallel.

THE ARCHITECTURE: A BROKER AND P MACHINES



The **broker** receives the query, contacts the machines and returns the answer. Every design must answer four questions:

1. What is split across the machines: the *objects* or the *inverted lists*?
2. Where does each piece of the index live, and how is it built?
3. Which machines does a query visit?
4. How are the partial results combined into the top K — *correctly*?

STRATEGY 1: PARTITION THE OBJECTS

- Split the N objects into P groups (e.g. at random). Machine M_p stores the feature vectors of its objects and builds a *local* MIF over them, using the **same pivots** R on every machine.
- Indexing is fully parallel: no communication, about $1/P$ of the work per machine.

M_1: objects $o_1, \ldots, o_5$

$r_1 \rightarrow$	$(o_5, 2)$			
$r_2 \rightarrow$	$(o_2, 2)$	$(o_4, 1)$	$(o_5, 1)$	
$r_3 \rightarrow$	$(o_1, 1)$	$(o_2, 1)$	$(o_3, 1)$	$(o_4, 2)$
$r_4 \rightarrow$	$(o_1, 2)$	$(o_3, 2)$		

M_2: objects $o_6, \ldots, o_{10}$

$r_1 \rightarrow$	$(o_7, 1)$	$(o_8, 2)$	$(o_9, 1)$	$(o_{10}, 2)$
$r_2 \rightarrow$	$(o_6, 1)$	$(o_7, 2)$		
$r_3 \rightarrow$	$(o_6, 2)$			
$r_4 \rightarrow$	$(o_8, 1)$	$(o_9, 2)$	$(o_{10}, 1)$	

Each local list is the part of the global list that belongs to the machine's own objects.

Variant: machines may even choose their own pivots; their scores are then incomparable, so results can only be merged on true distances.

OBJECT PARTITIONING: ANSWERING A QUERY

1. The broker sends q and $\tilde{L}(q, R) = (r_4, r_3)$ to **all** machines.
2. Each machine runs stages 1–3 on its own objects and returns its K best with their *true* distances.
3. **Result aggregation**: the broker keeps the K smallest true distances overall.

	candidates (points)	re-rank best $c = 2$	returns ($K = 2$)
M_1	o_1 (2), o_3 (2), o_2 (1), o_4 (1)	o_1, o_3	o_1: 1.11, o_3: 1.97
M_2	o_8 (2), o_{10} (2), o_6 (1), o_9 (1)	o_8, o_{10}	o_8: 1.33, o_{10}: 2.23
broker	merge by distance: o_1 (1.11), o_8 (1.33) — the true 2 nearest neighbours		

- **Is K per machine enough?** For the objects the machines re-ranked, yes: one in the overall top K is also in its own machine's top K (ties broken the same way everywhere). But merging cannot bring back a true neighbour already dropped in stage 1 or in a top-c cut.
- Cost: every query occupies all P machines; after a random split, each reads about $1/P$ of the postings on average.

STRATEGY 2: PARTITION THE INDEX BY PIVOT

- Each machine owns the *complete* inverted lists of n/P pivots, with postings from *all* objects.
- Indexing needs communication: once $\tilde{L}(o_i, R)$ is computed, each posting is sent to the machine that owns its pivot, so up to $N\tilde{n}$ postings cross the network.
- A query visits only the owners of q's $\tilde{n}$ pivots: at most $\tilde{n}$ machines, however large P is. (Feature vectors for re-ranking are stored separately, e.g. split by object ID.)

M_1 owns r_1 and r_3

$r_1 \rightarrow$	$(o_5, 2)$	$(o_7, 1)$	$(o_8, 2)$	$(o_9, 1)$	$(o_{10}, 2)$
$r_3 \rightarrow$	$(o_1, 1)$	$(o_2, 1)$	$(o_3, 1)$	$(o_4, 2)$	$(o_6, 2)$

M_2 owns r_2 and r_4

$r_2 \rightarrow$	$(o_2, 2)$	$(o_4, 1)$	$(o_5, 1)$	$(o_6, 1)$	$(o_7, 2)$
$r_4 \rightarrow$	$(o_1, 2)$	$(o_3, 2)$	$(o_8, 1)$	$(o_9, 2)$	$(o_{10}, 1)$

PIVOT PARTITIONING: ANSWERING A QUERY

1. The broker computes $\tilde{L}(q, R) = (r_4, r_3)$ and contacts the owners: M_2 for r_4, M_1 for r_3.
2. Each owner reads its list and sends the points of *every* posting:
   M_2 (r_4): o_1 +1, o_3 +1, o_8 +2, o_9 +1, o_{10} +2; M_1 (r_3): o_1, o_2, o_3, o_4, o_6 +1 each
3. **Result aggregation**: the broker adds up the points per object (o_1, o_3, o_8, o_{10}: 2; o_2, o_4, o_6, o_9: 1), keeps the best c, and re-ranks them after fetching their vectors.

Pitfall: one object's score is split across machines

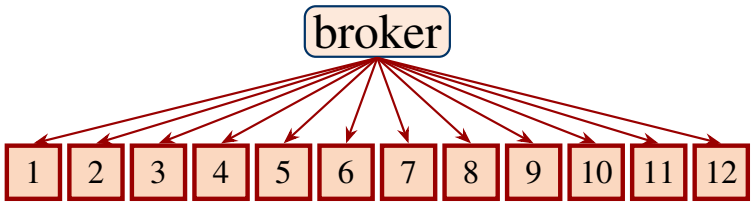
If each machine sent only its 2 best partial scores (M_2: o_8, o_{10}; M_1, all tied: o_1, o_2), the broker would credit o_1 with 1 point instead of 2, never see o_3, and rank o_8, o_{10} first. Partial scores must be *summed before* any cut-off.

Traffic: sending every posting is simple and correct, but each list read holds about $N\tilde{n}/n$ postings, so at scale this message volume becomes the bottleneck. An owner may add up its *own* contributions per object before sending (still correct), but may not drop objects.

THE STRATEGIES COMPARED ON 12 MACHINES

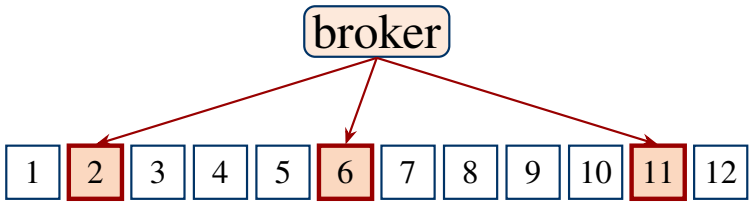
Hypothetical: 12 machines; 48 pivots (M_k owns $r_{4k-3}, \ldots, r_{4k}$); $\tilde{n} = 3$; q's pivots r_5, r_{22}, r_{41}.

Partition the objects



all 12 machines contacted

Partition by pivot



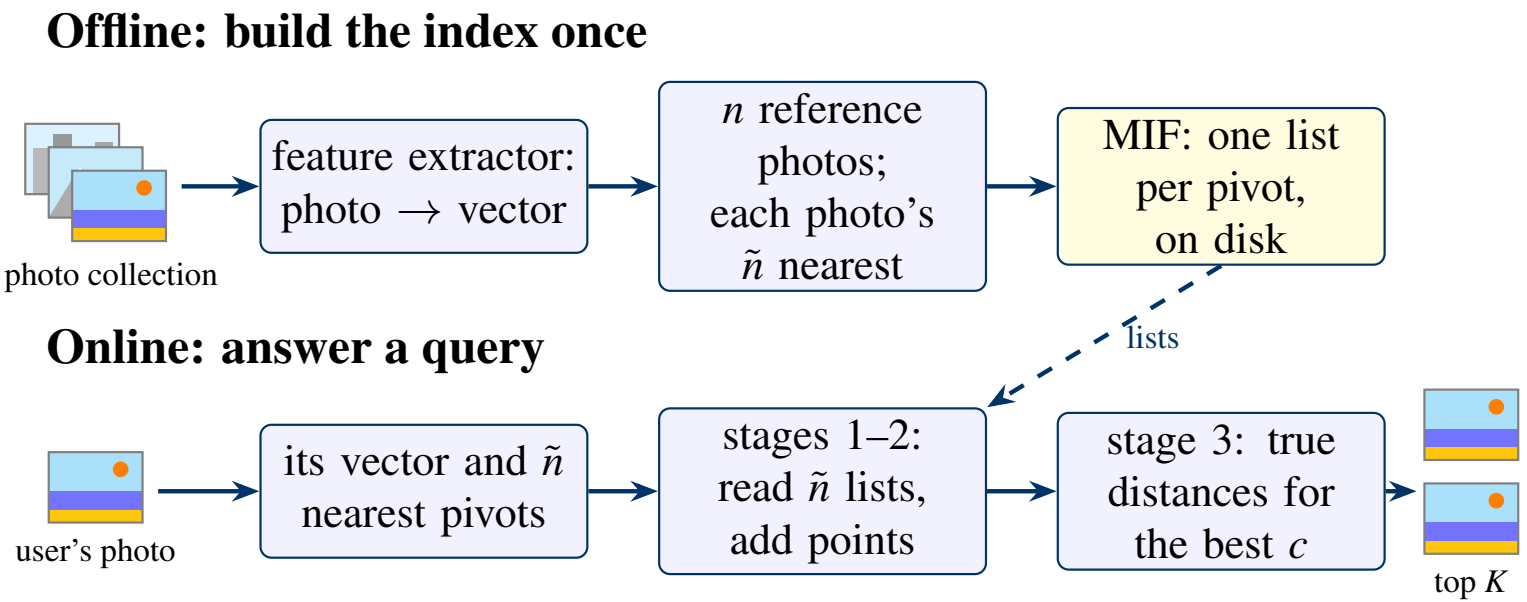
only the 3 owners contacted

	Partition the objects	**Partition by pivot**
indexing traffic	none	each object's 3 postings sent out
to the broker	120 results ($K = 10$)	$\approx 0.19\,N$ postings (3 full lists)
load	all machines, every query	owners only; hot spots possible
aggregation	keep K smallest distances	sum partial points, then select

Fewer machines per query is not free: pivot partitioning trades fan-out for traffic and load imbalance. ($0.19N = 3 \cdot N\tilde{n}/n$, lists of similar length.)

Practical perspective

END TO END: A SIMILAR-IMAGE SEARCH SERVICE



- **Feature vectors**: e.g. a colour histogram, or the output of a pretrained neural network (an *embedding*), chosen so that similar photos get nearby vectors.
- Every box is a step of this lecture: pruned lists (PBI), inverted lists (MIF), points (stage 2), exact re-ranking (stage 3); on many machines, add result aggregation.
- The same pipeline serves reverse image search, near-duplicate detection, "similar items" recommendations and semantic search over text embeddings.

EVALUATING AN INDEX: A SMALL EXPERIMENT

Measure on sample queries with known true neighbours (brute force): **recall@K**, **query cost** (distances and postings read: machine-independent stand-ins for time), **index size**, **build cost**.

Synthetic data: $N = 20\,000$ vectors in 32 dimensions (50 Gaussian clusters), $n = 256$ pivots, $K = 10$, 200 queries:

setting	recall@10	distances/query	postings read/query	index size
exhaustive scan	100.0%	20 000	—	—
MIF $\tilde{n} = 2$, $c = 200$	65.9%	453	450	40 000
MIF $\tilde{n} = 4$, $c = 200$	72.6%	456	1 534	80 000
MIF $\tilde{n} = 8$, $c = 200$	78.5%	456	6 772	160 000
MIF $\tilde{n} = 8$, $c = 50$	34.9%	306	6 772	160 000
MIF $\tilde{n} = 8$, $c = 800$	100.0%	1 053	6 772	160 000
prefix index, $\ell = 2$	23.3%	314	58 (group size)	20 000

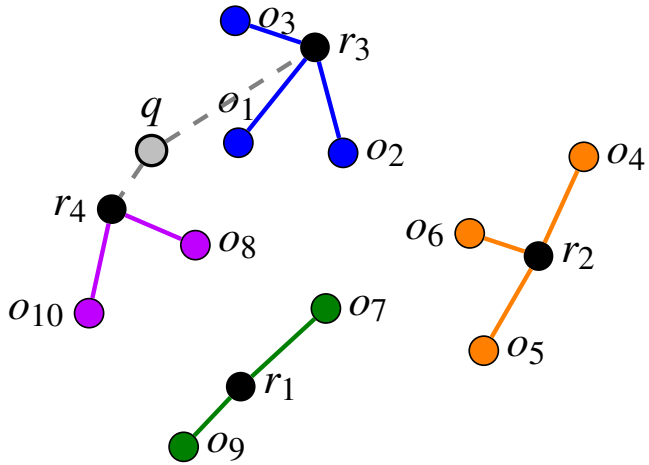
Each PBI index costs $Nn \approx 5.1$ million distances to build. Other seeds move recall by at most 3 points. Times depend on hardware and code, so counts are reported. Script: `arsiv/experiment_mif_tradeoffs.py`.

- No setting wins every column: approximate search is an **engineering trade-off** — choose the cheapest setting that meets the recall target. On this data the re-ranking budget c mattered most; longer lists helped less and multiplied the postings read.

BEYOND PBI: INVERTED-FILE (IVF) VECTOR INDEXES

How IVF works (e.g. in the Faiss library):

1. *Build*: cluster the vectors (typically k-means); store each vector in the list of its *nearest centroid*.
2. *Query*: scan only the lists of the n_{probe} centroids nearest to q; compute distances to those vectors (exactly, or approximately from compressed codes); return the best K.



Colour = list of the nearest "centroid"; dashed: q's two nearest.

Toy version: pretend $r_1, \ldots, r_4$ are centroids. q probes r_4, then r_3:

- $n_{\text{probe}} = 1$: $o_8, o_{10} \Rightarrow$ returns o_8, o_{10} (recall $1/2$)
- $n_{\text{probe}} = 2$: adds $o_1, o_2, o_3 \Rightarrow o_1, o_8$ (recall $2/2$)

Same principle, different method: like the MIF, IVF reads only a few inverted lists; unlike the MIF, its centroids are learned, each vector is stored once, and candidates are scored by distance, not by pivot order. **Other designs**: graph indexes such as HNSW walk a neighbourhood graph towards q; product quantization compresses vectors to fit more in memory. All trade recall against time and memory.

SUMMARY

- **Hashing** finds exact matches; **similarity search** needs different tools.
- **PBI** describes each object by the order of its nearest pivots: n distances per object, computed once.
- **Metric inverted file**: one list per pivot, $N\tilde{n}$ postings. A query reads only its $\tilde{n}$ lists (candidate generation), adds up points with missing pivots counted at position $\tilde{n}+1$ (approximate scoring) and re-ranks the best c by true distance: $n+c$ distances instead of N.
- **Prefix index**: one trie walk and, typically, one group read; compact and fast, but the prefix must match exactly.
- **Distribution**: partition the *objects* (all machines answer; merge true distances) or the *lists* (few machines, heavy traffic; sum partial points before cutting).
- **In practice**: measure recall@K against query cost, index size and build cost. IVF and graph indexes face the same trade-offs with different designs.

FURTHER READING

- E. Chávez, K. Figueroa, G. Navarro. Effective proximity retrieval by ordering permutations. *IEEE Trans. Pattern Analysis and Machine Intelligence* 30(9):1647–1658, 2008. (permutation-based indexing)
- G. Amato, P. Savino. Approximate similarity search in metric spaces using inverted files. *InfoScale* 2008. (metric inverted file)
- A. Esuli. PP-Index: using permutation prefixes for efficient and scalable approximate similarity search. *LSDS-IR Workshop at SIGIR* 2009, CEUR-WS vol. 480, pp. 17–24. (prefix index)
- R. Fagin, R. Kumar, D. Sivakumar. Comparing top k lists. *SIAM J. Discrete Mathematics* 17(1):134–160, 2003. (footrule with a location parameter)
- H. Jégou, M. Douze, C. Schmid. Product quantization for nearest neighbor search. *IEEE Trans. Pattern Analysis and Machine Intelligence* 33(1):117–128, 2011. (IVF with compressed vectors)
- Yu. A. Malkov, D. A. Yashunin. Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. *IEEE Trans. Pattern Analysis and Machine Intelligence* 42(4):824–836, 2020. (HNSW)
- M. Douze et al. The Faiss library. arXiv:2401.08281, 2024. (vector-search library)
- H. Mohamed, S. Marchand-Maillet. Scalable indexing for big data processing. In *Big Data: Algorithms, Analytics, and Applications*, Chapman & Hall/CRC, 2015. (lecture source)